

OSSP PRACTICAL Q8

8. Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind.

Develop a program that demonstrates Copy-on-Write behavior after a fork(). Analyze memory usage before and after modifying data in the child process.

CODE:-

```
#include <stdio.h> #include <stdlib.h>
```

```
int main() { int *a, *b, *c;
```

```
// malloc
```

```
a = malloc(5 * sizeof(int));
```

```
if (a == NULL)
```

```
{
```

```
    printf("malloc failed\n");
```

```
    return 1;
```

```
}
```

```
for (int i = 0; i < 5; i++)
```

```
    a[i] = i + 1;
```

```
printf("malloc: ");
```

```
for (int i = 0; i < 5; i++)
```

```
    printf("%d ", a[i]);
```

```
printf("\n");
```

```
// calloc
```

```
b = calloc(5, sizeof(int));
```

```
if (b == NULL)
```

```
{
```

```
    printf("calloc failed\n");  
    free(a);  
    return 1;  
}
```

```
printf("calloc: ");  
for (int i = 0; i < 5; i++)  
    printf("%d ", b[i]);
```

```
printf("\n");
```

```
// realloc  
c = realloc(a, 10 * sizeof(int));
```

```
if (c == NULL)  
{  
    printf("realloc failed\n");  
    free(a);  
    free(b);  
    return 1;  
}
```

```
a = c;
```

```
for (int i = 5; i < 10; i++)  
    a[i] = i + 1;
```

```
printf("After realloc: ");  
for (int i = 0; i < 10; i++)  
    printf("%d ", a[i]);
```

```
printf("\n");
```

```
// free  
free(a);  
free(b);
```

```
return 0;
```

```
}
```

8(b):-

```
#include <stdio.h> #include <stdlib.h> #include <unistd.h> #include <sys/wait.h>
```

```
int main() { int *data = malloc(5 * sizeof(int));
```

```
    for (int i = 0; i < 5; i++)
```

```
        data[i] = i + 1;
```

```
    printf("Before fork:\n");
```

```
    printf("Parent data address: %p\n", (void *)data);
```

```
    pid_t pid = fork();
```

```
    if (pid < 0)
```

```
    {
```

```
        perror("fork");
```

```
        free(data);
```

```
        return 1;
```

```
    }
```

```
    if (pid == 0)
```

```
    {
```

```
        printf("\nChild process:\n");
```

```
        printf("Child data address: %p\n", (void *)data);
```

```
        printf("Before modification: %d\n", data[0]);
```

```
        data[0] = 100;
```

```
        printf("After modification: %d\n", data[0]);
```

```
        printf("Child data address after modification: %p\n",  
            (void *)data);
```

```

    exit(0);
}
else
{
    wait(NULL);

    printf("\nParent process:\n");
    printf("Parent data address: %p\n", (void *)data);
    printf("Parent data[0]: %d\n", data[0]);
}

free(data);

return 0;

}

```

OUTPUT:-

UW PICO 5.09	File: p8a.c	Modified
<pre> } a = c; for (int i = 5; i < 10; i++) a[i] = i + 1; printf("After realloc: "); for (int i = 0; i < 10; i++) printf("%d ", a[i]); printf("\n"); // free free(a); free(b); return 0; } </pre>		

```
    data[0] = 100;

    printf("After modification: %d\n", data[0]);
    printf("Child data address after modification: %p\n",
           (void *)data);

    exit(0);
}
else
{
    wait(NULL);

    printf("\nParent process:\n");
    printf("Parent data address: %p\n", (void *)data);
    printf("Parent data[0]: %d\n", data[0]);
}

free(data);
```

^G Get Help	^O WriteOut	^R Read File	^Y Prev Pg	^K Cut Text	^C Cur Pos
^X Exit	^J Justify	^W Where is	^V Next Pg	^U UnCut Text	^T To Spell

```
[apple@apples-MacBook-Pro-2 practical % ./p8
malloc: 1 2 3 4 5
calloc: 0 0 0 0 0
After realloc: 1 2 3 4 5 6 7 8 9 10
```

```
apple@apples-MacBook-Pro-2 practical % ./p8a
Before fork:
Parent data address: 0x600002c70000

Child process:
Child data address: 0x600002c70000
Before modification: 1
After modification: 100
Child data address after modification: 0x600002c70000

Parent process:
Parent data address: 0x600002c70000
Parent data[0]: 1
```